· DDice ·

D&D; 5e (SRD) · Game Master’s Edition

— Contents —

Start Here 2

Setting Up 3

Rolling Dice 5

The Rules of 5e 9

Characters and Approvals 13

HP, Healing and Rest 19

NPCs 21

Quests 26

Standing 33

Activities and Quizzes 35

Scrolls, the Bot’s Voice, and Fixing What Was Said 45

Keeping the Server 46

Reference 48

Start Here

DDice runs the table. It rolls the dice, keeps every character sheet, runs fights turn by turn, posts quests, pays merits and remembers what happened. You talk to it in two ways. **Slash commands** begin with **/** and Discord shows every option as you type — **/roll stat:Strength** is the command **/roll** with its **stat** option filled in. **Typed shorthands** are plain messages the bot watches for — **1d20**, **str**, **hp +2** — for the things you do fifty times a night.

How to Read This Book

Commands are shown as you would type them. **@Skol** means pick a person from the list Discord offers; **name:**, **number:** and the like are options — type the label, then your answer. Where a line reads **(GM)**, only a Game Master can run it. The chapters follow the order you will meet things: rolling first, then your character, then fights, quests and standing.

`/help`	*(overview of all command groups)*
`/help category:start`	*(the new-player guide — the whole game in one page)*
`/help category:dice`	*(detail on a specific group)*
`/help category:rules`	*(the live table rules as the bot enforces them — the same words the books use, readable by everyone)*

*Anywhere a command asks for something that already exists — an NPC, a quest, a weapon, a tag, an activity, a category, a recovery schedule, an item someone is carrying — a **dropdown appears as you type**, and you can still type past it freely. Options that name something NEW (create, add) stay free text on purpose.*

Where the News Lands

A forum called **updates** carries one dated thread for every change the bot has had — titled by the day it landed, newest at the top — and every entry the bot has ever announced is there from the first. When something in this book looks different from what the bot does, the updates forum says what changed and when.

Setting Up

Everything the bot needs can be made in one command, and everything it made can be inspected and repaired in another. Two categories come into being: **DDice**, open to the table, and **DDice · Game Masters**, shut to everyone else. Nothing already set is touched, and a channel merely named for a job is taken up rather than doubled, so it is safe on a server set up by hand.

The One-Command Setup

`/gm check build`
the one-command setup: makes every channel and forum the bot needs in the two categories, points the config at each, then fills them with the audit shelves, pipeline books and tags. Needs Manage Channels, for you and for the bot (GM)

`(pickers)`
the check report offers channel pickers for unset configs — up to five at a time; a pick runs the real config branch, tags and books included (Manage Server required)

`/gm check run`
build anything missing — audit shelves, pipeline books, pipeline tags — and sweep every channel's permissions, repairing what it can; never moves or renames anything, and names what only you can grant. Surviving threads are adopted by id, never remade, so run it as often as you like, and after any update that adds a book (GM)

`/gm check status` — *(which channels are set, which await)*

`/gm check restart` — *(DELETE the whole setup and build it fresh — asks first, and the threads inside do not come back (GM))*

`/gm check order` — *(sidebar diagnostic — asks first: apply the plan order, or keep your layout and just report (GM))*

What Gets Made

Channel	What it is for
quest-board · forum	The board: a thread per posted quest.
quest-instances · forum	Each started quest opens its own instance here, for its party.
quest-chronicle · forum	A thread per adventurer — every quest they finished, told by the GM who ran it.
character-pages · forum	A page for each approved character.
the-fallen · channel	Those who did not come back, remembered by the table.
updates · forum	What changed for players — one dated thread per update.

Channel	What it is for
player-pdf · channel	The player book alone, reposted whenever it changes.
approvals · forum · GM	A thread apiece for sheets, trades, duels, lore and exports — each waiting on a GM.
roll-audit · forum · GM	A shelf per subject: rolls, checks, duels, items and the rest.
npc-pages · forum · GM	A thread per category, every NPC in it an entry inside — filter the sidebar to sort them.
npc-portraits · forum · GM	A thread per category, mirroring npc-pages. Caption an NPC's exact name for their face, or a bare order or category name for a shared one.
quest-planning · forum · GM	The pipeline books — concept, awaiting, approved, in progress, DMs, archived, completed.
quest-threads · forum · GM	A planning thread per quest, so the books keep the planning forum to themselves.
gm-quest-log · channel · GM	The machine's account of every finished quest — timings, events, payouts, final words.
gm-updates · forum · GM	What changed for GMs, and what each deploy mended — one dated thread each.
gm-digest · forum · GM	The weekly digest of what is waiting on a GM — one thread per week, titled by its Monday.
gm-feedback · forum · GM	Player feedback, one thread per category; players post with /feedback send.
gm-character-sheets · forum · GM	GM characters — the same five blocks and tags as the player forum, behind the GM category's eyes only.
campaigns · forum · GM	One thread per campaign — the arc, its linked runs, and where things stand.
question-bank · forum · GM	The quiz bank, a thread per category.
scroll-archive · forum · GM	Every prop written, kept.
memorial · channel · GM	The full record of a death — cause, deeds and standing.
backups · channel · GM	Where the nightly backup of everything is posted.
command-reference · channel · GM	The command books, reposted whenever they change.

GM Roles

`/config mechanics gmrole role:@Role`	*(add a GM role — several can be set)*
`/config mechanics gmrole role:@Role remove:true`	*(remove one)*
`/config mechanics gmrole role:@Role replace:true`	*(make it the only GM role)*
`/config mechanics gmrole`	*(list current GM roles)*

*Several GM roles can be set at once — holding any of them grants GM access. Anyone with the Discord **Manage Server** permission always counts as a GM, so you can never lock yourself out by mis-setting a role.*

Mechanics

`/config channels ruleset system:knightfall`

which rules this server plays by — Knightfall (five stats, blows decided by opposed rolls) or D&D 5e by the SRD (six abilities as modifiers, proficiency growing with level, attacks rolled against Armour Class). Set it BEFORE anyone makes a character: it refuses to change once sheets exist, because a sheet written for one system cannot be read as another. Run it bare to see which rules are in force

`/config mechanics statallowance points:15 minimum:1`	*(player build points (omit both to view))*
`/config mechanics hpbase base:3`	*(max HP = CON + this (default 2))*
`/config mechanics heal charges:3`	*(max heal charges for White Knights)*
`/config mechanics rest type:Short Rest hp:50% rerolls:0%`	*(% of max)*
`/config mechanics rest type:Short Rest hp:3 rerolls:1`	*(flat numbers)*

*Rest amounts come in two shapes, and the difference matters. A bare value **sets** the resource: **50%** puts them on half their maximum, **4** puts them on exactly 4 — even if that is fewer than they had. Prefix a **+** and it **adds** instead: **+4** gives four more HP, **+25%** gives a quarter of their maximum on top of what they have. Both cap at the maximum and both round down. **0%** leaves that resource untouched, and only the values you provide change.*

Rolling Dice

Type dice notation directly or use any prefix style (r, !r, !roll):

`1d20`	*(bare notation — no prefix needed)*
`r1d20+5`	
`r1d20+5 atk`	*(label shows in bold above result)*
`r1d20+5 atk` `[flavour text on next line]`	*(supports *italic* and **bold**)*

Advantage & Disadvantage

`ra1d20+5`	*(rolls twice, takes higher)*
`rd1d20+5`	*(rolls twice, takes lower)*

Guided Roll (/roll)

`/roll`	*(bare: a flat 1d20 — no stat, no modifier)*
`/roll stat:Strength`	*(pick a stat from the dropdown)*
`/roll stat:Wisdom mode:Advantage`	*(advantage / disadvantage)*
`/roll dice:2d6+3 label:damage`	*(custom notation instead of a stat)*
`/roll stat:Dexterity success_check:true label:sneak flavour:*slips into shadow*`	*(label + RP text)*
`/roll last:true`	*(recall your last roll in this channel)*

Every part of **/roll** is a dropdown or field — stat, advantage, success check, label and RP flavour — so nothing has to be memorised. Choose either a **stat** (rolls 1d20 + that stat from your sheet) or custom **dice**. The result uses the same card as a typed roll.

? Success Check Rolls

Prefix **?** instead of **r** to get a success outcome:

```
?1d20+5   ?ra1d20+5   ?rd1d20+5   ?rr   ?str
?wisa
```

Result	Outcome
Natural 20 / max face	Critical Success (gold)
Natural 1 / min face	Critical Fail (red)
Total 15+	Success
Total 10–14	Partial Success
Total 2–9	Fail

Critical Hits & Fails (all rolls)

Natural max on any die → gold label + result.
Natural 1 on any die → red label + result.

One Reroll Per Roll

A roll can be rerolled **once**. Whatever the second attempt says, it stands — you cannot keep spending rerolls on the same roll until the dice agree with you. This holds everywhere: typed rerolls, **/fight rr**, and the prompt inside an activity.

In a fight each side of an exchange gets its own second chance, so an attacker rerolling does not use up the defender's. A fresh exchange resets both.

Rerolls are still a resource — this is a limit on top of having one to spend, not instead of it.

Rerolls

`rr`	*(costs 1 token)*
`rra`	*(reroll with advantage)*
`rrd`	*(reroll with disadvantage)*

Shorthand: append a reroll set to a stat — **strrr**, **dexrra** (adv), **conrrd** (dis) — to reroll your last roll. A label may follow: **strrr atk**.

Game Master Rolls

`/gm roll notation:1d20+5`	*(public roll in channel)*
`/gm roll notation:1d20+5 label:perception`	*(with label)*
`/gm roll notation:1d20+5 secret:true`	*(secret — only you see the result; the roll audit records it either way)*
`gmr 1d20+5 perception`	*(typed chat shortcut for the same roll)*
`gmrs 1d20+5 stealth`	*(typed shortcut, secret — sent to your DMs)*

Calling for a Check

`/gm dc dc:14 stat:dex targets:@a @b npcs:Orc`

call a check. Players get a roll button each — a press rolls their own d20 (+stat, signature honoured) vs the DC and lands in the audit; named NPCs roll instantly on the card. dice: swaps the stat for any notation (2d6+1); mode: sets how they roll it — advantage, disadvantage, or a bare d20 (no stat, no signature); modifier: adjusts totals ±20; secret:true hides the DC (you get it privately) and reveal:@a whispers it to chosen players when they press — per-player sight on one check; on_fail: and on_success: each mark their NEXT roll — advantage or disadvantage — and the mark is GENERAL: it rides the character, needs no fight to be set, and is spent by whatever they roll next, anywhere — a chat !r, /roll, an activity, any fight action, or another check (specialist abilities keep their chosen mode). flat stays a fight mark: their next fight action as a bare d20. fail_damage: and success_damage: cost HP on that outcome, sheet and fight kept in sync. Nat 20 always passes, nat 1 always fails; pressed buttons go dark. Works mid-fight and with /fight auto npconly — full-auto runs start to finish, so a check lands after it. success_flavour: and fail_flavour: stay hidden on the card and are revealed with each roller's result; success_sanction: and fail_sanction: shift a stat by decree (pattern dex-1 or lck+2, ±5 at most, floor 0) — the stakes show on the card, the words wait for the outcome

`/gm dc target:@a npcs:Orc`

leave dc: out and a WRITING WINDOW opens — the check as a form: the scene · the check line (e.g. "dex 14 hidden", or "2d6+1 8 bare adv -2") · on success · on failure · targets. Each outcome box takes a tag line first — [adv] [dis] [bare] [dex-1] [-3hp], any order, either outcome — then the words the players read. Targets picked on the command carry into the form; the last box adds any you missed. Naming dc: keeps the fast path with every option instead

`/gm dc … hold:true skipfail:true`

bind the check to the fight: the target's actions wait on the card, and a failed check passes their turn

Buttons and Targets

`/button roll dice:2d6+1 dc:9 reason:… for:@a once:true`
plant a roll: dice OR a stat, an optional DC, the reason shown above it, and who it is for — anyone if left open. Pressed inside a quest, it joins that quest's timeline and the GM log (GM)

`/button group stat:wis dc:12 reason:…`
one check the whole party rolls — each presses once, the message keeps the tally, and Call it closes the scene with how many got through (GM)

`/target create name:Barricade stat:str dc:12`
plant something to swing at — no sheet, no roster. Each hit rolls publicly, then asks YOU whether it falls; secret:true asks in the GM channel instead (GM)

`/target list` — *(what still stands in this channel (GM))*

Overrides

`/gm override skip [reason:]` — *(pass the current turn in this channel's fight — announced as a GM ruling)*

`/gm override interject user:@a amount:2 note:…`
bend a player's roll mid-fight, any combination: force their stat, adjust the total (±10, stacks), set advantage/dis/flat, or DECLARE THE DIE (1–20) — the one lever that rewrites a roll already cast; the total recomputes around their modifier and every automatic honors the declared face; announced, audited (GM)

`/gm reroll target:@p stat:dex mode:dis flat:true`
interrupt and correct a mistaken roll by decree — wrong stat, wrong footing, or a bonus they never had. If they hold the pending attack or defence in a live fight here, the corrected roll REPLACES it in the fight (resolve sees the new number); otherwise their last roll in this channel is rerolled on the corrected terms. flat:true forces a bare d20; signature advantage is honoured unless flat; carried riposte/fumble modifiers drop — the GM is re-declaring the terms. NO reroll token is spent and the player's own once-per-roll right stays untouched. Mirrors to the audit as a GM correction

The Roll Audit

`/config channels rollaudit channel:#gm-rolls` — *(mirror every roll — raw input, result and jump link)*

`/config channels rollaudit test:true` — *(send a test mirror, report problems)*

`/config channels rollauditforum forum:#roll-audit`
split the mirror into books — player rolls, GM rolls, NPC rolls, NPC say, and the Scrolls archive; rerunning adds any missing book

`(the books)`
the audit forum keeps a shelf per subject: Player Rolls · GM Rolls · NPC Rolls · NPC Say · Scrolls · Called Checks · GM Overrides · Duels · Advancement · The Fallen · Items. Re-run the setup after an update and any new shelf is added without touching the old ones

`/config channels rollauditforum disable:true` — *(back to the single audit channel)*

The roll-audit mirror records every roll, in every channel, with nothing skipped. *Prefix rolls, stat shorthand, success checks, rerolls, heals,* ***/roll****,* ***/gm roll*** *and every fight roll — plus Game Master rolls, including secret* ***gmrs*** *ones, so Game Masters are accountable to one another.*

*A Game Master rolling as an NPC with **/npc roll** is logged under their own name, tagged with the NPC they spoke as — the roll itself goes out through the NPC's webhook, so the audit is the only place it ties back to a person.*

*Rolls the bot makes for itself are recorded too, attributed to the fighter and tagged **auto**: auto-pilot attacks, defences and reroll answers, initiative at the start of every fight and whenever an NPC joins mid-fight, every roll of a full **/fight auto**, and demo bouts. Rolls typed inside the audit channel itself are mirrored as well — a secret **gmrs** there goes to the Game Master's DMs, so without the mirror it would leave no record at all.*

*Use **/config channels rollaudit test:true** to check it works. Set the channel's Discord permissions so only Game Masters can view it. When NPC stats are hidden, manual and auto fight cards mask the stat and modifier — the audit's NPC book keeps the full card — roll line, real HP and all five stats revealed.*

◈

The Rules of 5e

A server set to **D&D 5e** plays by the SRD. Only what the SRD releases freely is built in; anything from the Player's Handbook alone — most subclasses, backgrounds, feats — your GMs add through the same custom routes the bot already has.

Set it with **/config channels ruleset system:dnd5e** BEFORE anyone makes a character. It refuses to change once sheets exist, because a sheet written for one system cannot be read as another.

Making a 5e Character

One command builds the sheet: **/dnd create**. Give the six scores, and it works out the rest — modifiers, proficiency from level, and hit points from the class die and CON.

```
/dnd create name:Sir Aldric str:16 dex:14 con:15      (a level 5 fighter in chain mail)
  int:10 wis:12 cha:8 class:Fighter level:5 ac:16
  saves:str, con skills:athletics, perception
```

The class sets the hit die — d12 for a Barbarian, d10 for Fighter, Paladin and Ranger, d8 for Bard, Cleric, Druid, Monk, Rogue and Warlock, d6 for Sorcerer and Wizard. Leave **ac:** out and it is 10 plus your DEX modifier. Saves and skills are written as you say them, separated by commas; an unknown skill is refused with the eighteen listed.

The sheet then reads in 5e terms: each score with its modifier beside it, hit points, Armour Class and proficiency on the face of it, and any saves or skills you are trained in underneath.

Abilities and Modifiers

Six abilities — **STR DEX CON INT WIS CHA** — each a score from which a modifier is taken. Ten is average and adds nothing; every two points either way is one point of modifier.

```
score 8                                   (modifier −1)
score 10 or 11                            (modifier +0)
score 14                                  (modifier +2)
score 18                                  (modifier +4)
```

`score 20`	*(modifier +5)*

Proficiency

A proficiency bonus is added to anything a character is trained in — attacks with their weapons, saving throws of their class, skills they have taken. It grows with level.

`levels 1 to 4`	*(+2)*
`levels 5 to 8`	*(+3)*
`levels 9 to 12`	*(+4)*
`levels 13 to 16`	*(+5)*
`levels 17 to 20`	*(+6)*

Hit Points

At first level, the class hit die plus the CON modifier. Every level after adds half the die (rounded up) plus the CON modifier again. A d10 class with CON 16 has 13 at first level and 22 at second.

Attacks and Armour Class

To attack, roll a d20 and add your modifier and your proficiency. You are trying to reach the target's **Armour Class** — that is just a number, so nobody rolls to defend. Meeting the AC exactly is a hit. A natural 20 always hits and is a critical; a natural 1 always misses, whatever the total.

Damage is the weapon's dice plus the ability modifier. A critical rolls the **dice** twice and leaves the modifier alone — so a 1d8+3 hit that crits is 2d8+3, never 2d8+6.

Levels, Hit Dice and Rest

A level is not just a bigger number. **/dnd levelup** raises it and everything follows: the hit points arrive full, another hit die is added, and the proficiency bonus rises when it is due. A GM can level someone else with **user:**, or jump straight to a level with **to:**.

You carry one **hit die** per level — the die your class rolls. A **short rest** spends one: roll it, add your CON modifier, and heal that much. A **long rest** fills you to full and hands back half your dice, rounded down, never fewer than one.

`!srest`	*(spend a hit die and heal)*
`!lrest`	*(full hit points, half your dice back)*
`/dnd levelup`	*(gain the next level)*
`/dnd levelup user:@Bo to:5`	*(a GM setting someone to level 5)*

A level 5 Fighter with CON 15 has 44 hit points, +3 proficiency and five d10 hit dice; a short rest heals between 3 and 12 of them.

Fighting in 5e

Combat runs through the same commands as everywhere else — **/fight start**, **/fight atk**, the turn order, the initiative — but a blow is settled differently. Nobody rolls to defend: the attack is measured against the target's Armour Class and resolves on the spot.

`/fight atk target:@Bo`	*(d20 + ability modifier vs their AC)*
`18 vs AC 15`	*(a hit — damage follows at once)*
`natural 20`	*(always hits, and the weapon dice are rolled twice)*
`natural 1`	*(always misses, whatever the total)*

Damage is the weapon's dice plus the ability modifier, and the card shows the working — **1d8 [5] +3** — so nobody has to take the total on trust. A weapon with no dice set swings 1d8.

Dying

At nought hit points a 5e character is dying, not dead. **/dnd deathsave** rolls a d20 — ten or better is a success, under ten a failure, a natural one counts as two failures, and a natural twenty is not a save at all — they come back on a single hit point. Three successes and they are stable; three failures and a GM records it with **/gm kill**, or a healer argues with it first.

Any healing that lifts them above nought clears the tally, wherever it came from — a spell, a rest, or a GM's hand.

Saves, Skills and Initiative

A saving throw or a skill check rides on **/roll**: **/roll save:dex** or **/roll skill:stealth**. The ability modifier is added, and your proficiency bonus on top when your class has that save or you have taken that skill — the card says so when it applies. All eighteen skills offer a dropdown as you type.

`/roll save:con`	*(a Constitution save)*
`/roll skill:perception`	*(a Wisdom (Perception) check)*
`/roll skill:athletics mode:Advantage`	*(with advantage)*

Going first: on a 5e server you add your DEX **modifier**; on Knightfall you add the whole DEX score — the same command, the right arithmetic for the rules in force.

Weapons

Set what a weapon rolls with **/dnd weapondice slot:1 dice:1d12**, and an attack uses it for damage instead of the default 1d8. Clear it and it falls back.

Conditions, and Being Hard to Kill

All fifteen SRD conditions are known. **/dnd condition add:prone** takes one on, **remove:** lifts it, **clear:true** lifts them all, and a GM can name a **user:** or an **npc:**. They bend the dice on their own — a prone creature rolls at disadvantage and is easier to hit, an invisible one the reverse — and one of each cancels out, as the rules say.

At nought hit points a character is dying, not dead. **/dnd deathsave** rolls a d20 — ten or more is a success, under ten a failure, a natural one counts as two failures, and a natural twenty brings them back on a single hit point. Three either way settles it, and any healing above nought clears the tally.

`/dnd deathsave`	*(roll while dying)*
`/dnd temphp amount:8`	*(temporary hit points — spent first, never stacked)*
`/dnd inspiration grant:true`	*(a GM granting it)*
`/dnd inspiration use:true`	*(spending it for advantage)*

Damage types are honoured where a sheet declares them: write **fire, immune poison, x2 acid** and a blow is halved, spared or doubled, and the card says which it did. Temporary hit points are spent before real ones.

Extra Attack

A Fighter swings twice from fifth level, three times from eleventh and four from twentieth; a Barbarian, Paladin, Ranger or Monk swings twice from fifth. The turn is held until the action is spent, and the card says which swing you are on.

Magic

A caster's slots follow from their class and level, so the bot works them out. **/spell slots** shows what is left, with a filled circle for a slot in hand and an empty one for a slot spent, along with the spell save DC (8 + proficiency + the casting ability) and the spell attack bonus.

`/spell slots`	*(what magic you have left today)*
`/spell cast level:3 name:Fireball`	*(spend a third-level slot)*
`/spell cast level:1 name:Bless concentration:true`	*(and hold it together)*
`/spell concentration`	*(what you are holding)*
`/spell concentration drop:true`	*(let it go)*

Three patterns are known. **Full casters** — Bard, Cleric, Druid, Sorcerer, Wizard — read the usual table. **Half casters** — Paladin and Ranger — come in at second level and read it at half theirs. A **Warlock's** pact magic is a handful of slots all at one level, and they return on a **short** rest rather than a long one.

Clerics, Druids, Wizards and Paladins prepare spells: the ability modifier plus their level, or half their level for a Paladin. **/spell prepare add:** readies one, **drop:** lets one go and **clear:true** starts again — the list is capped at what you may hold, and it tells you when it is full. A long rest returns every slot.

Concentration is tracked. Casting a second concentration spell lets the first go, and when a blow lands on a caster the card asks for the save: ***CON, DC 10 or half the damage taken, whichever is harder****.*

Saves and Skills

A saving throw means rolling a d20 and adding your modifier. If your class is good at that kind of save, you add your proficiency too. A skill check is the same, against a DC the GM sets. All eighteen SRD skills are known, each tied to its ability: Athletics to STR; Acrobatics, Sleight of Hand and Stealth to DEX; Arcana, History, Investigation, Nature and Religion to INT; Animal Handling, Insight, Medicine, Perception and Survival to WIS; Deception, Intimidation, Performance and Persuasion to CHA.

What is live today: the rules above are implemented, and a 5e server uses 5e abilities and hit points. Combat still resolves the Knightfall way until the fight engine is wired to attack-versus-AC — that work, a 5e character sheet, classes and levels, and spellcasting are each still to come.

◈

Characters and Approvals

Reading & Sharing Your Sheet

`/char create ...` — *(see Making Your Character — builds the whole sheet)*

`/char view show` — **`/char view show user:@player`**

`/char export` — **`/char export format:Image`**

`/char export format:Parchment image`
the sheet on parchment, in the scroll font — for display; Discord strips an image's weave on re-upload, so use the Parchment PDF or Text block to travel

`/char export format:Parchment PDF`
the same parchment as a PDF — the one file that survives Discord re-uploads, sheet woven after its EOF

`/char export format:Career PDF`
the career record as a native-text parchment PDF, career woven after its EOF — survives Discord and imports back through /char import

`/char export format:Summary`
the career record — rank, merits, renown, dice history, pack, quests, lore — over a paste-able [TTRPG SUMMARY] block

`/char import`
hand an exported sheet OR career block to this server — a Parchment PDF (the carrier that survives Discord), a pasted block, or bare for a paste box; a GM approves. Careers move merits and renown to the imported totals and add dice history, items and lore — sheet, rank, quests, tags untouched

`/char submit` — *(resend your sheet after a rejection)*

With sheet approval switched on, an export goes to the GMs first. Running **/char export** posts the sheet block into the approval channel with **Release to player** / **Decline** buttons; the player sees only a private note that it has been sent. When a GM releases it, the block arrives by DM — or back in the channel they exported from, if their DMs are closed. Exporting is not an edit: it never changes a sheet's approval state and never stops anyone rolling. Exporting again replaces the previous request, so the channel holds one

live entry per player. GMs export straight away, for themselves and for anyone else, and on servers not using approvals nothing changes.

The Pages of a Sheet

Beyond the stat block, a character accumulates things worth keeping. Each page can be read for yourself or for someone else with **user:**.

`/char view show full:true`	*(the card and every page below, on one post)*
`/char view inventory`	*(what they are carrying)*
`/char view standing`	*(merit and renown, and where each came from)*
`/char view rollhistory`	*(every natural die this character has rolled)*
`/char view rollhistory sides:6`	*(the same for a different die)*
`/char lore`	*(write your lore and send it to the Game Masters)*
`/char view showlore`	*(read approved lore)*

Inventory holds roleplay items — things an activity handed over, or a Game Master gave out by hand with **/char give user:@a item:A tarnished silver key**. Each carries a number, so **/char take user:@a id:3** removes one. Giving and taking are Game Master-only.

Standing puts total merit and renown at the top, then lists every movement beneath it — the quest, activity or Game Master award that caused it, the amount, and how long ago. Merits are a lifetime tally that only climbs; renown is a currency that is spent again.

Roll history counts every natural die a character has ever rolled, drawn as a bar chart with the natural 20s and 1s marked. It follows the character, not the player, and covers every roll they make — typed, slash, in a fight or in an activity.

Lore opens a writing box, pre-filled with whatever you wrote last time. It goes to the same channel character sheets do, with Approve / Reject buttons; a rejection asks the Game Master why and passes the reason back. Only approved lore shows on **/char view showlore** — before that it says it is waiting, or why it was turned down. Rewriting retires the old request and sends a fresh one.

Nothing here is required. A character works perfectly well with an empty inventory, no lore and no renown — these are for tables that want to track more.

Your Thread

With a character forum set, every approved character gets a thread of their own there — the sheet first, then inventory, lore, standing, titles and associations as posts beneath it — kept current by the bot, so the sheet in your thread changes the moment a stat does. A button on the page asks a GM to update your lore document; everything else writes itself.

Your Roll Card

How much of your sheet appears when you roll is up to you.

`/char profile card style:Full`	*(the whole sheet)*
`/char profile card style:Compressed`	*(one line of stats)*
`/char profile card style:Off`	*(plain text rolls)*

Compressed puts everything on a single line under the roll — **STR 5 · CON 5 · DEX 4 · WIS 4 · LCK 2 · HP 8/8 · RR 2/2** — which keeps a busy channel readable without hiding where a modifier came from.

*A **stat roll always shows something**, even with your card Off: **1d20+4** says nothing about which stat it was or where the 4 came from, so those fall back to the compressed line. Choosing Compressed is respected rather than overridden — the bot never upgrades you to Full. Plain dice rolls honour Off completely.*

Saved Sheets

`/char profile save slot:vault`	*(keep a copy of your sheet)*
`/char profile load slot:vault`	*(put it back)*
`/char profile saves`	*(what you have saved)*

Loading a save is checked like any other change — an old snapshot cannot smuggle in a spread that breaks the current point allowance.

Profile

`/char profile on`	*(enable embed, max HP + rerolls)*
`/char profile off`	*(disable embed, plain text rolls)*
`/char profile show`	*(preview card without rolling)*

Sheet Import

Paste an exported sheet into any channel the bot watches.

`[paste sheet into channel]`	*(your own exported sheet, back onto this server)*
`[paste sheet] @player`	*(Game Master importing for another player)*

Heroes

Hero is a **status**, not a class. A Game Master grants it, and it sits alongside whatever class and knight order a character already holds — a Hero can be a Green Knight Vanguard.

`/char hero user:@a value:true`	*(Game Master grants it)*
`/char hero user:@a value:false`	*(and takes it away)*
`/char signature user:@a stat:Strength`	*(a Hero rolls that stat with advantage)*

It used to occupy the class slot, so a Hero could be nothing else. Anyone whose class said Hero has been moved across automatically and their class slot freed. Removing Hero status clears the

signature stat with it.

The Queue

Everything waiting on a decision, in one place — character sheets, lore, merit trades and quest applicants, with jump links to each. It also surfaces any background job that is failing.

/gm queue	*(what is waiting)*

Pending lore had no listing anywhere else, so a submission whose queue post was deleted was effectively invisible. This finds it.

Approvals

/config channels approvals channel:#sheet-approvals	*(new sheets need GM sign-off)*
/config channels approvals list:true	*(every sheet still waiting — from the database)*
/config channels approvals disable:true	*(turn approval off)*
/config channels approvalforum forum:#gm-approvals	*(one forum, a thread per approval type — sheets, trades, duels, lore, exports)*
/config channels approvalforum disable:true	*(back to the single channel — posted items keep working)*

With an approval channel set, a player's new sheet is posted there for sign-off, pinging every GM role, with Approve / Reject buttons. Until approved they can't roll, heal or take fight actions — and once approved, their whole sheet can only be changed by a GM. Sheets a GM creates or edits skip the queue, and sheets made before approval was enabled keep working until someone edits them.

*Every way a player can write to their own sheet goes to the queue: **/char create**, **/char set** (any field — stats, order, class and weapons alike), **/char weaponemoji**, **/char profile load** and pasting an exported sheet. Building a character one **/char set** at a time is not a way past a GM. Editing a sheet that is still pending retires the old request and posts a fresh one, so the channel holds one live entry per player rather than a pile of stale ones. A rejected sheet can still be edited by its owner — that is how they fix it — and each edit sends it straight back for another look.*

*A sheet that is **still waiting** can be edited by its owner — spot a mistake before a Game Master gets to it and you can fix it, which retires the old request and posts a fresh one. Only an **approved** sheet is frozen to its owner.*

*Pressing **Reject** opens a box asking **why**. The note is optional, but it travels with the decision — it is written onto the request in the approval channel, sent to the player with the rejection, and shown again if they try to roll before fixing it, so nobody is left guessing at what to change. Declining an **export** asks the same way.*

*A **rejection is never the end of it.** A rejected sheet stays editable by its owner, so the player fixes whatever the GM objected to with **/char set** or **/char create** and rejoins the queue on their own. If they believe it was right as it stood, **/char submit** sends it again unchanged. There is no limit — a sheet can go back and forth as many times as it takes, and each resubmission retires the previous request so the channel still holds one live entry per player. The rejection notice spells out both routes, so nobody is left waiting on a GM to do it for them.*

*A declined **export** works the same way — running **/char export** again puts a fresh request in front of the GMs.*

*Sheets are accepted from **any channel the bot can read** — an ordinary text channel, a thread, a forum post, an announcement channel, or the text chat inside a **voice or stage channel**. Wherever it came from is recorded with the request, so the decision notice can find its way back there if the player's DMs are closed.*

*The queue lives in the database, not in a Discord message. **/config channels approvals list:true** shows every sheet still waiting — who, how long ago, and which channel it came from — even if the request was never posted, was deleted, or landed somewhere nobody reads. If the bot cannot post to the approval channel it says so on that list and pings the GM roles in the channel the sheet was submitted from, so a player is never left locked out in silence.*

Items & Pages

/char give user:@a item:A tarnished silver key \ note:Cold to the touch	*(Game Master only)*
/char take user:@a id:3	*(by the number in their inventory)*
/char edit user:@a id:3 item:Silver key note:Cold	*(reword an item)*
/char view show user:@a full:true	*(their card, and every page below it at once)*
/char view standing user:@a	*(merit and renown, and where each came from)*
/char view rollhistory user:@a	*(every natural die they have rolled)*

*Items are free text — whatever a story calls for. Activities can hand them over too, and a player reads them back with **/char view inventory**. Lore submitted with **/char lore** arrives in the sheet approval channel with Approve / Reject buttons, and a rejection asks you why.*

Character Pages

Point the bot at a **forum** and every approved character gets a post of their own — somewhere for lore, art and notes. The link then appears on **/char view show** and in search results.

/config channels charforum channel:#character-sheets	*(the forum)*
/config channels gmcharforum channel:#gm-character-sheets	*(GM-only forum for GM sheets — same five blocks and tags, behind the GM category's permissions)*

(character forum)

one thread per character: Sheet → Inventory → Lore → Standing → Titles → Associations → Dice, all bot-kept (dice hourly: d2–d20, averages, nat extremes); tags = order · class · Fallen · Hero; threads are staff-typed (players ask a Moderator/Expeditioner — a notice in every thread says so, with a Request lore update button — link your Google Doc and it lands in the GMs' Lore Docs approvals); GM sheets live in the GM-only forum, auto-created on update where the GM category exists

/char page user:@a	*(make one now)*
/char page user:@a thread:#their-thread	*(or link a post that already exists)*

`/char page user:@a unlink:true`	*(forget it — the thread itself is untouched)*

*Pages are made when a sheet is **approved**. If the forum is missing or the bot cannot post there, the approval still goes through and the Game Master is told what went wrong — a broken forum must never block a character. If a thread is later deleted, the next attempt makes a fresh one rather than linking to nothing.*

Searching the Roster

`/gm search order:Green Knight`	*(everyone in an order)*
`/gm search class:Vanguard hero:true`	*(combine filters)*
`/gm search order:— none set —`	*(find who still needs one)*
`/gm search who:NPCs`	*(or narrow to one side)*
`/gm search name:mat fallen:true`	*(part of a name, including the dead)*

Results read in the same order as everything else, with each character's page linked beside them. The fallen are left out unless asked for.

Preset Tags

```
/char tag assign user:@player tag:Hero of Kalidale
/char tag remove user:@player tag:Hero of Kalidale
/char tag list user:@player
```

Custom Tags

```
/char tag custom action:Create emoji:[any emoji]
name:MyTag
/char tag custom action:Delete name:MyTag
/char tag custom action:List
```

For server emojis: type \:emojiname: in chat to get the full ID.

Heroes & Signature Stats

Hero is a status a Game Master grants, not a player choice — players who try to set it are refused. It sits alongside whatever class and knight order a character already holds, and both player sheets and NPCs can carry it.

`/char hero user:@a value:true`	*(grant Hero to a player sheet)*
`/char signature user:@a stat:str`	*(designate their signature stat)*
`/npc hero name:Aldric stat:dex`	*(Hero NPC with a signature stat)*

A Hero may have one **signature stat** with **5 or more** points. Every roll using that stat — typed shorthand, **/roll**, and both attack and defence in fights, manual or automatic — is made with **advantage**, marked on the roll card. If the stat later drops below 5 the advantage simply stops applying until it is restored; an explicitly requested disadvantage still wins.

HP, Healing and Rest

Game Master HP & Rerolls Targeting

```
!hp @user +5    !hp @user -3
!hp +5 @user    !hp -3 @user
!rerolls @user +1    !rerolls @user -1
```

Game Master Rest Targeting

```
lrest @user   srest @user   hpfull @user   hphalf
@user
```

Restoring Resources (/gm heal)

`/gm heal user:@a`	*(full HP — the default)*
`/gm heal user:@a restore:Everything`	*(HP, rerolls and heal charges)*
`/gm heal user:@a amount:Half`	*(restore half of maximum)*
`/gm heal user:@a amount:Add value:3`	*(add 3, capped at max)*
`/gm heal user:@a amount:Exact value:1`	*(set to an exact figure)*
`/gm heal npc:Goblin`	*(one NPC)*
`/gm heal npc:all`	*(every NPC at once)*

One command for every restore. Works on a player or an NPC (or **all** NPCs), and HP changes sync straight into any active fight. NPCs only have HP; heal charges are skipped for anyone who isn't a White Knight with WIS 5+.

global: restores everyone at once instead of naming a target — **Players**, **NPCs**, or **Everyone** together. It takes the same **amount** and **restore** options as a single target, so **/gm heal global:Everyone amount:Half** puts the whole server on half HP, and **/gm heal global:Players restore:Everything** hands every character HP, rerolls and heal charges back. Pick exactly one of **user**, **npc** or **global**.

*Unlike scheduled recovery, a global heal does **not** skip players out on a quest. A Game Master typing this has decided to heal the room, and a silent exclusion mid-session would be a nasty surprise. Heal charges still only reach White Knights with WIS 5+, and every change syncs into any fight already running.*

Scheduled Recovery

A server can run **any number of named schedules**, each with its own timing and its own strength. A light top-up every few hours and a full recovery overnight can sit side by side.

```
/config mechanics autorest action:Add or update      (half HP, rounded down, every 6h)
name:Breather \
  hours:6 hp:50% rerolls:0% heal:0%
```

```
/config mechanics autorest action:Add or update      (everything back, once a day)
name:Full Recovery \
  hours:24 hp:100% rerolls:100% heal:100%

/config mechanics autorest action:List               (what is set, and when each next
                                                      falls)

/config mechanics autorest action:Run now            (fire one immediately)
name:Breather

/config mechanics autorest action:Pause              (stop it without deleting it)
name:Breather

/config mechanics autorest action:Remove             (delete it)
name:Breather
```

Amounts use the same tokens as **/config mechanics rest**. A bare value **sets** the resource — **100%** full, **50%** half, **4** exactly four. A **+** prefix **adds** instead: **+4** is four more HP, **+25%** a quarter of their maximum on top. **0%** leaves it alone. Everything caps at the maximum and rounds down, so half of 11 HP is 5. A typo is refused when you set the schedule rather than quietly doing nothing at three in the morning.

Anyone on a quest that is in progress is skipped by every schedule. Their HP, rerolls and charges stay exactly where they are until the quest is completed, so being out in the field costs something. Applicants, and members of quests still open or already finished, are restored as normal. Heal charges only go to White Knights with WIS 5+, as everywhere else.

Give a schedule a **channel:** and each run is announced there, naming what it did, who was restored and who was left out in the field.

Because schedules are **named and independent**, the counters need not share a cadence. A common pair is HP overnight and the charge counters twice a day:

```
/config mechanics autorest action:Add name:Night     (HP once a day)
hours:24   hp:100% rerolls:0% heal:0%

/config mechanics autorest action:Add name:Charges   (rerolls and heal charges every
hours:12   hp:0% rerolls:100% heal:100%               12h)
```

***0%** means leave it alone, not set it to nothing — which is what lets one schedule move HP and another move only the charges. Both run on their own clocks without interfering.*

NPCs rest on the same schedule. They have only HP — no rerolls, no heal charges — so the HP setting is the whole of it for them, and any change syncs straight into a fight already running.

*Two groups are left alone, players and NPCs alike: anyone **out on an active quest**, and anyone **standing in a fight**. Healing a fighter on a timer would undo an exchange partway through. The **fallen** are skipped entirely — a rest is not a resurrection. Each reason is reported separately, so it is clear who was passed over and why.*

Each schedule carries its own clock in the database, so a restart or redeploy can neither skip a cycle nor fire one early. Adding a schedule, or resuming a paused one, starts its count from that moment.

```
/config mechanics autorest action:List               (every recovery schedule)
```

`/config mechanics autorest action:Add or update name:Breather \ hours:6 hp:50% rerolls:0% heal:0%`	*(a light top-up)*
`/config mechanics autorest action:Run now name:Breather`	*(fire one immediately)*
`(when they land)`	*(rests fall ON the hour — a 12-hour rest set at 14:37 next falls due at 02:00, not 02:37)*

`/config mechanics autorest action:Who is excluded`
who the next rest will pass over, and why — the fallen, anyone on an active quest, anyone in an active fight, and which quest or fight is holding them

NPCs

NPC Management

`/npc create name:Cave Orc str:14 con:10 dex:4 wis:2 lck:1 order:Black Knight`	*(full stat block)*
`/npc create name:Mystery Figure`	*(name only — stats added later)*
`/npc create name:Mystery Figure str:14 con:10`	*(fill stats in over time)*
`/npc manage copy name:Goblin new_name:Goblin 2`	*(duplicate an NPC, fresh HP)*
`/npc show name:Goblin`	*(full stat block for one NPC)*
`/npc hero name:Goblin stat:str`	*(make an NPC a Hero with a signature stat)*
`/npc hero name:Goblin remove:true`	*(strip Hero status)*
`/npc list`	*(shows stats and current HP)*
`/npc list category:Bandits`	*(only one category's NPCs)*
`/npc delete name:Aldric Vane`	

*Stats are optional — an NPC can be registered by name (and given an avatar) with no stats at all, then statted up later. Re-running **/npc create** with the same name updates only the fields you supply and leaves the rest untouched.*

NPC HP persists between fights. Knocked-down NPCs (0 HP or less) are left out of new fights until restored.

NPC HP & Healing

`/npc hp name:Goblin value:3`	*(set exact HP)*
`/npc hp name:Goblin`	*(omit value — full heal)*
`/npc heal names:all`	*(fully heal every NPC)*
`/npc heal names:Goblin, Orc`	*(fully heal the listed NPCs)*

Speaking as an NPC

```
/npc say name:Cave Orc speech:Halt! Who goes there?        (speech — in quote marks)

/npc say name:Cave Orc action:raises its axe               (action — italic emote)

/npc say name:Cave Orc action:raises its axe               (both, stacked)
  speech:Halt!

/npc say name:Cave Orc                                     (opens a writing box)
```

Works in threads and forum posts too. *A thread cannot own a webhook, so the NPC's voice is created on the parent channel instead and every message is routed back into the thread it was called from — several threads under one channel share the same NPC webhook.*

Posts as the NPC through their webhook — their name and avatar, no dice rolled. Fill **action**, **speech**, or both: the action is italicised and the speech is wrapped in quote marks, stacked on separate lines. Leave both blank and a **writing box** opens with roomy multi-line fields — easier for longer roleplay. Use **raw** to post exactly as typed.

Rolling as an NPC

```
/npc roll name:Cave Orc notation:1d20+8
label:strike
  flavour:The orc lunges

/npc reroll name:Cave Orc [roll:adv]                       (reroll that NPC's last roll in this
                                                           channel — spends one of its LCK
                                                           tokens)

/npc create name:Aldric Vane str:8 con:6 dex:10
wis:4 lck:2

/npc list [category:] [compact:true]
    the roster, paged — fifteen with full stats a page, or sixty names a page in compact; ◀ ▶ turn the
    pages and a button swaps the view. Big rosters no longer flood the channel
```

Posts via webhook — appears as the NPC with their name and avatar. DDice uses **one shared webhook per channel**, so any number of NPCs can speak there without meeting Discord's limit of fifteen webhooks per channel.

```
/config mechanics cleanwebhooks
    reclaim spare NPC webhooks — DDice keeps one per channel, so anything else it owns there is
    a leftover and can be freed
```

Bot requires Manage Webhooks permission in the server.

NPC Pages

Point the bot at a forum with **/config channels npcforum** and every NPC gets a page there — their statblock, their categories, their face and whatever lore is written — kept up to date as they change. The page wears their categories as Discord's own **tags**, so the sidebar filter sorts them for you.

Say what kind they are when you make them and the category comes into being on the spot: **/npc create name:Garrick category:Enemy**. The first enemy you write brings **Enemy** into being; everyone after joins it. Add more later with **/npc category assign** — an NPC can belong to several, which is why each has one page rather than one per category.

`/config channels npcforum channel:#npc-pages`	*(where the pages live)*
`/npc create name:Garrick category:Enemy`	*(made, filed, and written up)*
`/npc category assign npc:Garrick category:Vendor`	*(a second category — a second TAG on their thread: tap tags in the forum to filter NPCs natively)*
`/npc manage sync [name:]`	*(write up everyone already made)*

Twenty tags is Discord's limit for a forum. Beyond that, NPCs are still given pages — they simply go untagged rather than the whole thing failing. Deleting an NPC takes their page with them.

Setting NPC Avatars

`/config channels npcchannel #channel`	*(set the NPC portrait forum — a thread per category)*

1. Admin runs **/config channels npcchannel #channel** to set the portrait forum.
2. Game Master uploads an image to that channel with the NPC name as the message text.
3. Bot adds a checkmark reaction to confirm. 4. Re-upload with the same name to update.

One Face for a Whole Order — or a Whole Category

Shared faces come from **tags**, not names. Post a picture in the portrait forum captioned with a bare **order** or **category** name and every member without a personal portrait wears it. Who wears what: their own face first, then their order's, then the first of their categories that has one. The old **Order | Name** pipe convention is retired — a pipe in a name is just a name.

`White Knight`	*(caption → shared face for every White Knight)*
`Merchant`	*(caption → shared face for every Merchant-tagged NPC)*
`Garrick Vale`	*(caption → that one NPC's personal face)*

Hidden Stats

`/config mechanics npcstats enabled:true`	*(reveal NPC stat blocks — hidden by default)*

*NPC roll cards hide STR/CON/DEX/WIS/LCK, the roll modifier and HP totals by default. Players see the name, order, the final roll total, the exact damage each hit deals, and a condition (unhurt / wounded / badly hurt / near death / down) instead of numbers — so the fight reads clearly without revealing what an NPC can take. Reveal everything with **/config mechanics npcstats enabled:true**. NPC management commands are Game Master-only regardless.*

NPC Records

An NPC keeps the same records a player does. **/npc sheet** shows the lot on one page — stats and HP, standing, inventory, lifetime roll history and lore.

`/npc sheet name:Cave Orc`	*(the whole record)*
`/npc give name:Cave Orc item:… note:…`	*(hand them something)*

`/npc take name:Cave Orc id:1`	*(take it back)*
`/npc npclore name:Cave Orc text:…`	*(write their story)*
`/npc delete name:Cave Orc`	*(remove them entirely)*

Their dice count too. Every roll the auto-pilot makes for an NPC — attacks, defences, reroll answers, initiative — goes into that NPC's lifetime tally, so a long-running villain builds a record of their own luck exactly as a player does.

Merit and renown work on an NPC the same way they do on a character, so an NPC can hold standing in the world, be paid in merit, or carry the reward for a job.

Categories

`/npc category create name:Bandits`	*(make a grouping)*
`/npc category assign name:Cave Orc category:Bandits`	*(file an NPC under it)*
`/npc category remove name:Cave Orc`	*(take it out)*
`/npc category list`	*(every category and who is in it)*
`/npc category delete name:Bandits`	*(remove the grouping)*

Temporary NPCs — Enemies for One Fight

Most enemies exist for one fight and should leave when it ends. A **temporary** NPC is a throwaway: no page, no roster seat, no record — it fights, and when the fight stands down, however it ends, every temporary NPC is swept away. Make them one at a time, clone a stock enemy by the handful through an encounter preset, or keep one that turned out to matter.

`/npc create name:Goblin str:3 con:2 dex:3 wis:1 lck:1 temp:true`	*(a throwaway — no page, swept after the fight)*
`/fight addnpc npc:Goblin, Goblin 2`	*(into a running fight, or name them on /fight start)*
`/encounter preset save name:Ridge bandits merits:2 npcs:Goblin x3`	*(a saved scene; starting an encounter from it clones Goblin three times, temporary)*
`/npc temp list`	*(every throwaway still lying about)*
`/npc temp keep name:Goblin 2`	*(make one permanent — it gains a page)*
`/npc temp clear`	*(sweep them all away now)*

***/gm check rehearse** opens a private sparring thread against a **Sparring Dummy** the bot runs — a temporary NPC of its own, so every button can be tried before the table sees it, and nothing is left behind.*

The Library

Rather than typing a monster in every time it appears, keep it. The library holds monsters and spells for the whole server, and **/library summon** brings one out as an NPC ready to fight — several at once, numbered, if you name a **count:**.

`/library srd what:Both`	*(load the set that ships with the bot)*
`/library list [search:]`	*(what the library holds)*
`/library show name:Goblin`	*(read one entry in full)*
`/library summon name:Goblin count:4 as:Scout`	*(four Scouts step out, ready to fight)*
`/library import file:monsters.txt`	*(read in your own)*
`/library forget name:Goblin`	*(remove an entry)*

The set that ships here is drawn from the **System Reference Document**, which is released under Creative Commons — thirty monsters from bandits and goblins to an ancient red dragon and a lich, and twenty spells from Fire Bolt to Wish. It is a working core rather than the whole document; anything else you want, you import.

Importing your own

One entry a line. The name comes first, then any fields you like, separated by pipes, in any order. Anything the parser does not recognise is kept as a note rather than thrown away, so your own wording survives.

`[MONSTER] Goblin | ac 15 | hp 7 | attack +4 | damage 1d6+2 | str 8 dex 14 con 10 int 10 wis 8 cha 8 | cr 1/4`	*(a monster)*
`[SPELL] Fireball | level 3 | school evocation | range 150 ft | 8d6 fire, DEX save halves`	*(a spell)*
`# lines starting with a hash are ignored`	*(a comment)*

Attach a **.txt** up to 400 KB, or paste entries into **paste:** separated by **;;**. A monster needs at least an **ac** and **hp**; a spell needs a **level** (0 for a cantrip). Anything already known is skipped and named, unless you pass **replace:true**. The reply says what was read, what was skipped and what was refused, so nothing fails quietly.

Only SRD material ships with the bot. Anything from the Player's Handbook, Monster Manual or Dungeon Master's Guide is yours to enter for your own table — the import command is there for exactly that.

Monsters

A monster is made from the four numbers any statblock leads with: **/npc manage create5e name:Orc ac:13 hp:15 attack:5 damage:1d12+3**. Its abilities are optional and sit at 10 unless you say otherwise. From then on it fights by its own numbers — the attack bonus off the block rather than a class it does not have, and the damage dice with their flat bonus.

`/npc manage create5e name:Orc ac:13 hp:15 attack:5 damage:1d12+3 str:16 con:16`	*(the SRD orc)*
`/fight start players:@a npcs:Orc`	*(and it is in the initiative)*

`/npc say name:Orc speech:...`	*(as ever, with its own face)*

Before this, a monster on a 5e server had no Armour Class of its own and one was guessed from a Knightfall stat — which put most of them around AC 7, so nearly everything hit. Give a monster its AC and that is settled.

◈

Quests

GMs create quests with lore, objectives, details and rewards. Quests are posted as a message with an Apply button and also appear on the board. Players apply, a Game Master approves the party, and on completion merits are auto-awarded while other rewards are listed for the Game Master to hand out.

For Players

`/quest board`	*(list quests (filter: open/active/completed/all))*
`/quest show number:1`	*(full details of one quest)*
`/quest roster number:1`	*(see applicants and the party)*
`/quest party apply number:1`	*(apply to join — or tap Apply on the post)*
`/quest party withdraw number:1`	*(leave or cancel your application)*
`/quest record user:@player`	*(one player's finished quests — what they have been on)*

Quests are auto-numbered for easy recall, e.g. ***#001-Goblin Cave*** *(repeatable quests keep the name and get a fresh number each time). Party size is a hard cap or a suggestion — the Game Master chooses with* ***hard_cap****, and* ***force:true*** *on approve overrides a cap. Point a quest at a thread with* ***/quest runchannel*** *and its completion rewards are announced there.*

Your Party's Thread

When a run starts it opens its own thread in **quest-instances** with the whole party pulled in. That thread is where the quest happens: fights fought there log themselves on the quest's timeline, the Game Master's rally pings everyone in it, and the ending is written up there before it goes anywhere else. A player holds one seat at a time — you cannot be approved onto a second quest while a run is still live.

Encounters

An encounter is a session on the fly — no board, no applications. The Game Master names it, ticks who is in it from the roster and opens it; it gets a thread of its own in quest-instances like any run, and pays out like one. The seats are the Game Master's: there is no Join button, so if you want in, ask. An encounter does not count as your one quest, so a party already out on a quest can still be jumped by bandits.

`/encounter list`	*(every encounter running now, and where)*

Campaigns

A campaign is the long arc — quests and encounters under one name. Its card shows the premise, what is done and your own part in it; the Game Master's notes and status stay on their side.

/campaign list	*(every campaign on the server)*
/campaign show campaign:The Falconer's War	*(the premise, what is done, and your part)*

The Chronicle and the Review

When a run is completed, the Game Master's telling of it lands in your own thread in **quest-chronicle** — one thread per adventurer, every quest they finished, told by whoever ran it — and the summary is linked on your standing page. The run's thread then carries a review button: a score out of five and a line or two, for the Game Masters alone to read. **/quest record** lists what you have been on.

Writing a Quest

/quest create
bare, a five-field writing window opens — name, objectives, lore, details and rewards as full paragraphs. Numeric options (merit_reward:, party_size:, hard_cap:, gm_style:) ride along. from:N seeds the window from an existing quest as a fresh draft — a template copy, unlike instance. With name: given, everything stays inline as before

/quest edit number:1
the same window, prefilled with what stands — the natural editor. Saving updates the board embed and renames the board and planning threads. Numeric options apply directly without the window

/quest create name:Goblin Cave objectives:... merit_reward:2 party_size:4 hard_cap:true *(create a quest)*

*Every **number:** option across /quest autocompletes — start typing a number or part of a name and the matching quests appear as "#012 — Goblin Cave (open)".*

The Pipeline

/config channels questforum channel:#forum *(make the quest board a forum — one thread per posted quest (Admin))*

/config channels questthreads channel:#forum
optional split: per-quest planning threads open in this secondary forum instead, and the pipeline stage tags ride along with them — so the planning forum holds only the six books and the DM roster. Disable to fold threads back (existing threads stay where they are either way) (Admin)

/config channels questinstances
a forum where every STARTED quest opens its own thread for party — the DM and each member pulled in by mention, the clock and reminders following them there. Optional: without it a quest runs in its run channel as before

/config channels questplanning channel:#gm-forum

a private GM forum: /quest create opens a planning thread there with the full quest record, and every application and lifecycle event mirrors into it; /quest post is the flip that opens the public board thread. Setup also creates the seven pinned pipeline BOOKS — Concept, Awaiting Approval, Approved, In Progress, DMs Available, Archived, Completed — the same way the roll-audit forum builds its books, plus the matching stage tags. Every quest keeps one index entry (name, GM, party, links to its planning thread, board post and the party's room) that moves between books as it goes; a live run shows how long its clock has been going, anything sitting still for days says so, Completed entries carry the run counter, and the DMs book is the roster (Admin)

`/quest stage number:1 stage:approved`
move a quest through the hand-set stages — or just press the advance button at the bottom of its planning thread (→→, one press each; at Approved it points to /quest post). Posting, archiving and completing re-tag automatically — each stage line in the planning thread REPLACES the last, so the walk never piles up; posting leaves exactly one line: the board link

`(buttons)`
each application in the planning thread carries Approve / Kick buttons; at the Approved stage the thread's button becomes Post to board — GM-gated, same checks as the commands

`/config mechanics questspinoff enabled:true`
the board becomes listings: approvals stage, and one launch births a run carrying the whole staged group

`(who runs it)`
the first GM to approve an applicant becomes that quest's DM — /quest handoff passes it to another GM. When approvals reach the party size, that DM is pinged once in the planning thread

`/quest dm style:... brief:...`
your DM card on the DMs Available roster — style, a short brief, available:false to step back, and a running tracker: quests written · parties guided to completion. Re-rendered on every card change, quest creation and completion

`/quest post number:1`
post the quest with an Apply button. With a quest forum configured this opens the quest's own thread on the board — applications, party changes, start, pause, public notes and completion all mirror in, and the thread archives when the quest completes. Pass channel: to post plainly instead

`/quest archive number:1`
take a quest off the board — applications close, the Apply button drops, the board thread locks and archives; /quest post re-lists it

Applicants and the Launch

`/quest party approve number:1 user:@a force:true` *(approve · under spin-off this STAGES them on the listing until you launch · force past a hard cap)*

`(one quest at a time)`
a player may hold only one seat: approving someone already on a live run is refused, and a launch leaves them an applicant rather than double-booking them

`/quest party kick number:1 user:@a` *(remove a member or applicant)*

`/quest run start number:1` *(lock the party, mark in progress)*

`/quest run start number:1 (listing)`

the launch: the whole staged group becomes one run with its own thread and clock — or press on the post

/instance add name:X [run:] user:@a *(seat a player on a run — name autocompletes your listings, run blank = the latest active one)*

(naming a run)
paste the thread's own name if you like — the number prefix and "Run 001 …" suffix are ignored. In a run's own thread you may leave name: blank entirely

/instance kick/rally/note/pause/resume/complete/show/thread
the in-play nine, addressed by name and run — each forwards into /quest, so gates and behaviour are identical

(run names) *(every launch is christened "<Quest> Run 001 <GM>" — the thread and the ledger wear the same name)*

/quest thread number:1 · /quest thread all:true
open a quest thread for one that hasn't got one. Use it for quests that started before the instance forum was set — all:true catches every running quest at once. Anything that already has a room is left alone

/quest handoff number:1 gm:@them
pass a quest to another GM — they become its DM. The board, the book, the planning thread and the party's room are all told

Several GMs, the Same Adventure

A quest holds one party on one clock, so two Game Masters cannot share a quest number. **/quest instance** makes a separate run of the same adventure: it copies the writing — name, lore, objectives, details, rewards, merit and party rules — and leaves everything else fresh. A new number, an empty party, a clock at zero and its own log.

/quest instance number:1 [label:Blackfen party]
run your own copy of an adventure. The copy keeps the ORIGINAL's number and adds which run it is — #002.2-Testing the waters · Blackfen party — so the board reads as one story with several tables. Fresh party, clock at zero, you as its DM

/quest instance number:1 gm_style:Roleplay-focused *(and advertise how you run it)*

Three Game Masters can run the same adventure at once, each with their own party, channel, clock and summary. Completing one does nothing to the others. The board marks them: One of 3 separate runs of this adventure.

Running a Quest — the Clock

Starting a quest with **/quest run start** begins a stopwatch. From then on the bot posts a public time check in the quest's run channel **every 15 minutes**, and **on the hour** a recap of everything that happened during it. Set the channel first with **/quest runchannel**, or there is nowhere for them to go.

/quest run start number:1 *(the clock begins)*

/quest run note number:1 text:They bribed the gatekeeper kind:Roleplay *(mark a moment)*

`/quest run timeline number:1`	*(the whole log so far)*
`/quest run pause number:1`	*(stop the clock, keep the time)*
`/quest run resume number:1`	*(carry on where it left off)*
`/quest run complete number:1`	*(stop, award, and write it up)*

A timeline reads back like a ship's log:

```
` 0h 00m` Quest begins — 4 on the party
` 0h 15m` Time check — 0h 15m
` 0h 17m` Cave Orc speaks
` 0h 44m` Artorius wins the fight
` 1h 00m` Hourly recap — 3 events
```

Combat and roleplay log themselves. A fight ending in the quest's run channel, or an NPC speaking there through **/npc say**, is attached to whichever quest is running in that channel. Anything else you want on the record goes on with **/quest run note**, taggable as roleplay, combat or a plain note.

Pause keeps everything. *The time already run is banked and the clock stops — a paused quest gets no reminders and logs nothing. Resuming picks up at exactly the same figure. Both counters live on the quest itself, so a restart or redeploy mid-session resumes rather than starting the count again.*

`/quest runchannel number:1 channel:#thread`	*(set where it runs & rewards)*

`/quest rally number:1 [message:] [here:true]`
call the party together — pings every member in the quest thread (or in this channel with here:true), with your message and a line naming their DM

`/quest run note number:1 text:... public:true`	*(log a moment — planning-thread mirror by default; public:true posts it in the board thread too)*

`/quest run recap number:1`
drafts a "previously on…" from everything logged — private to you; add post:true to read it to the party (GM)

`/quest npc number:1 name:Orc`	*(attach the NPCs a run involves (remove:true to detach) — they appear on the completion run record)*

`/quest runs number:1`
the run ledger for a quest and all its instances: how many times it has been completed, and each run's date, GM, party and NPCs

The Quest Summary

On **/quest run complete** the clock stops and the whole run is written up: who ran it, how they run a table, how long it took, who was on the party, and the full timeline. Set where it goes with **/config channels questlog channel:#chronicle**.

`/config channels questlog channel:#chronicle`	*(where finished quests are written up)*

`/config channels questlog disable:true` *(stop posting them)*

The summary is then **linked on every party member's standing page — /char view standing** and **/char view show full:true** both list the quests a character has finished, each one a link straight to its write-up, with how long it took and how long ago it was.

Ending a Run

`/quest run winddown number:1`
the story is over but the rewards are not settled: the run stays live, its party is released to the rests and free to join other quests, and anyone kicked now is still paid at completion. resume:true puts it back (GM)

`/quest run complete number:1` *(finish — auto-award merits, list rewards)*

`/quest run complete number:1 summary:…`
add your telling — it lands in each adventurer's thread in quest-chronicle; timings, events and payouts go to the GM-only gm-quest-log instead

`/quest run log number:1 text:…`
write or rewrite the tale afterwards — it is mirrored into every adventurer's own chronicle thread, and rewriting edits each copy in place (GM)

`(↩ Reopen, on a completion message)`
five minutes to take a completion back: payout reversed, this run's titles revoked, party released; fix it and complete again (GM)

`(reviews)`
a completed run posts a review button in its own thread; `/quest show` then carries the count, the average and the latest few lines (GM)

`/quest archive number:1 summary:…`
the final shelf. Blank summary: opens a box for the final word — it goes on the record, into the run's thread and the GM quest log, and marks the campaign card. A completed quest stays completed on everyone's record (GM)

`/quest delete number:1`
remove a quest permanently: its board thread or post and its planning thread (applications and notes included) and the party's room are deleted with it — and any instances of it go the same way, threads and all. A fight still running in the room is stood down first. If the quest is live, the confirmation says so before you commit: the clock stops and the party is released. Its number returns to the pool: the next quest created takes the lowest free number, reborn with a clean history (the run ledger keeps counting for DM credit but no longer answers to that number)

GM Style

A quest can advertise how its Game Master runs a table, so a player knows what they are applying to. Set it on **/quest create** or per instance; it shows on the quest card, the board post and the final summary.

Tag	What it promises
Mechanics-focused	rules, rolls and tactics to the fore
Roleplay-focused	character and conversation to the fore
Mixed elements	a bit of both

Tag	What it promises
Combat-heavy	expect fighting
Puzzle & investigation	problems to work out
Sandbox	the players decide where it goes

Encounters — Sessions on the Fly

`/encounter start name:… merits:2 campaign:…`
a session on the fly: no board, no applications. The bot lists every living character with HP and what holds them; tick who is in it and press Open the encounter — it opens its own thread in quest-instances, the party pulled in by mention. No Join button, ever (GM)

`/encounter add user:@a · /encounter remove user:@a`	*(the seats are yours: seat or remove a player, run in the encounter's thread (GM))*
`/encounter end summary:… title:…`	*(pays out, writes the chronicle, posts the review button — the ordinary completion (GM))*
`/encounter list`	*(every encounter running now and where)*

`/encounter preset save name:… merits:2 npcs:Goblin x3`
a saved scene; `/encounter start … preset:` runs it with the merits set and the monsters summoned as temps; `preset list` · `preset remove` (GM)

*An encounter left quiet for six hours pings the GM roles in its room; it never ends itself. A preset's **npcs:** are summoned from the 5e library where there is one — and where there is not, an NPC of that name is cloned as temporary fighters, so **Goblin x3** works on a Knightfall server from a single Goblin you made once.*

Campaigns — the Long Arc

`/campaign create name:… description:…`
the long arc: quests and encounters under one name, with a GM-only thread and a live card. add · remove · note · status · recap · complete; `campaign:` on `/quest create` and `/encounter start` links at birth (GM)

`/campaign add campaign:… quest:2`	*(link a quest that already exists — old ones too)*
`/campaign note campaign:… text:…`	*(a beat that happened between runs)*
`/campaign status campaign:… text:…`	*(where things stand — the present, not the history)*
`/campaign recap campaign:…`	*(a previously-on across every linked run)*
`/campaign complete campaign:…`	*(close the arc)*
`/campaign show campaign:…`	*(players see the premise, what is done and their own part; GMs see everything)*

Stalled Work and the Weekly Digest

`/gm check stalled`
what is waiting on a GM: winding-down runs, quiet encounters, staged parties never launched, idle campaigns, reviews this week (GM)

Once a week the same report is posted on its own, as a thread in the **gm-digest** forum titled by the Monday of the week it covers — so nothing depends on somebody remembering to ask.

Feedback Rooms

`/button feedback`	*(plant a feedback button in this channel (GM))*
`/feedback category add|remove|list`	*(the rooms feedback lands in — seven to begin with; add your own and its thread opens at once (GM))*
`/feedback category list`	*(every room feedback can land in)*
`/feedback category remove name:X`	*(retire a custom room (GM))*

Standing

A milestone-style progression system. Merits are a lifetime tally a Game Master awards; ranks are named tiers with merit thresholds. The bot tracks progress and flags eligibility, but every promotion is decided by a Game Master.

Merits

`/standing merit view`	*(your merits, rank, and merits to next rank)*
`/standing merit view user:@player`	*(view another player)*
`/standing merit leaderboard`	*(top earners on the server)*
`/standing merit add user:@player amount:2`	*(award merits — Game Master)*
`/standing merit remove user:@player amount:1`	*(take merits away — Game Master)*
`/standing merit set user:@player amount:10`	*(set an exact total — Game Master)*
`/standing merit history user:@player`	*(a player's merit timeline)*
`/standing merit history`	*(recent server-wide merit activity)*

Ranks

`/standing rank list`	*(all ranks and their thresholds)*
`/standing rank add name:Knight threshold:5`	*(create or update a rank — Game Master)*
`/standing rank add name:Squire threshold:0 order:0`	*(order sets junior→senior — Game Master)*

`/standing rank promote user:@player rank:Knight`	*(set a player's rank — Game Master)*
`/standing rank strip user:@a`	*clear their held rank — merits and renown stay, so they can re-claim whatever they still qualify for (GM)*
`/standing rank eligible`	*(who has met a threshold but isn't promoted — Game Master)*
`/standing rank remove name:Squire`	*(delete a rank — Game Master)*

/standing merit view *shows current merits and exactly how many more are needed for the next rank, e.g. "Knight · 7 merits · 8 to Paladin". Every merit change is recorded — quest rewards show the quest name, manual changes show "by GM" — so* ***/standing merit history*** *answers "who earned what, and when". Removing a rank doesn't change players who already hold its label.*

Titles & Associations

A title is a name earned — granted by a Game Master, or carried away by every survivor of a quest that offered one, stamped with where it came from. An association is a company, order or cause a character stands with. Both show on the character's page.

`/standing title list user:@a`	*(the titles they hold)*
`/standing association list group:The Falconers`	*(who stands with a company)*
`/standing title grant user:@a title:… source:…`	*(award a title — it shows on their page, their NPC sheet if they are one, and their roll card (GM))*
`/standing title revoke user:@a title:…`	*(take one back (GM))*
`/standing association add user:@a group:… note:…`	*(the company, order or cause they stand with (GM))*
`/standing association remove user:@a group:…`	*(they part ways (GM))*
`/quest run complete number:1 title:…`	*(every survivor earns the title, stamped with the quest it came from (GM))*

Renown

Renown is not a currency. It is a running tally of how a character **stands in the world** — what they have done, who has noticed, and how far their name carries. It is earned from quests, encounters and activities, and it is not meant to be traded away for goods. Someone with high renown is **known**; that is the whole of it.

A Game Master can adjust it either way when the story calls for it — a reputation can be damaged as well as built — but it is a record of standing rather than a purse.

`/standing renown view` `/standing renown view user:@player`	
`/standing renown leaderboard`	*(who is best known)*
`/standing renown history`	*(where a standing came from)*
`/standing renown gain user:@a amount:5 reason:Cleared the Sunken Vault`	*(Game Master)*

`/standing renown loss user:@a amount:3 reason:Disgraced at court`	*(Game Master)*
`/standing renown set user:@a amount:0`	*(Game Master)*

Every change is logged with its reason, so **/standing renown history** answers how a reputation was built and where it was lost.

Merit

Merit is the earned measure of service — awarded by a Game Master, accumulated across quests and activities, and the thing rank thresholds are set against. Unlike renown, **merit is tradeable**: it can be passed between players, and potentially to and from NPCs, as payment, tribute, a debt settled or a favour bought.

`/standing merit give user:@a amount:2 reason:A debt settled`	*(offer some of your merit)*
`/standing merit trades`	*(what is still waiting on a Game Master)*
`/standing merit cancel id:3`	*(withdraw one — your own, or any as a Game Master)*

No merit moves until a Game Master signs it off. An offer is held and posted to the sheet approval channel with Approve / Refuse buttons; a refusal asks the Game Master why and passes the reason back. Both parties are told when it lands, and it is announced in the channel where it was offered so the table sees the trade happen.

Your balance is checked twice — when you offer, and again when a Game Master approves. If you have spent the merit in between, the trade is voided rather than pushing anyone into the red.

Renown says who you are in the world. Merit is what you have earned and may hand on. A character can be widely known and hold no merit at all, or quietly hold a great deal.

◈

Activities and Quizzes

An activity is a minigame you write for your server: the bot narrates, asks for rolls, branches on the results and loops until someone stops. Fishing, foraging, a gauntlet in the training yard — whatever you script. Only a Game Master can write one; whether players can **start** one is a setting.

Running One

`/activity run name:Fishing [player:@someone]`
start an activity here. Name a player and it runs for THEM — their name on it, their buttons, their score — which is how you set a quiz for someone to sit (GM only). Several runs can go at once in one channel; each answers only to its own player

`/activity list`	`/activity show name:Fishing`	*(what exists, and read it back in full)*
`/activity stop`		*(abandon the run here)*

`/activity demo which:Kalidale Lore [player:@someone]`
play a built-in activity — the fishing tale or a five-question lore quiz. Name a player and it runs for THEM, so you can hand someone a quiz to sit; nothing is awarded either way

`/activity create`	*(write one — a paste window, or file: a .txt for long scripts (Game Master))*
`/activity set name:X scene:find field:Roll value:dex`	*(tweak one line (Game Master))*
`/activity delete name:X`	*(remove it (Game Master, asks first))*
`/config mechanics activities players:true`	*(let players start them too)*

/activity demo plays a ready-made fishing game so you can see the whole system working before writing anything: a difficulty check that loops with a different excuse each time, four sizes of catch chosen by the roll, a gauntlet per size, natural 20s and 1s, and buttons to keep going or head back. It awards **nothing** — no renown, no merits, no items — so it is safe to play with.

One Run Each

An activity run belongs to the person who started it. Several people can play in the same channel at once, each with their own prompts, their own progress and their own rewards — and one person stopping or wandering off does nothing to anyone else. Every post is tagged with whose run it is, and a button only answers for its owner.

`/activity run name:Fishing`	*(start your own run)*
`/activity stop`	*(end yours — a Game Master with none running can clear the channel)*

Answering a Scene

Press the button, or **type the stat and add your own flavour after it** — both roll the same thing, but typing lets you say what your character is doing.

`wis I survey the reeds where the current slows`	*(rolls WIS, prints your words)*

A typed roll only answers the scene if the stat is one that step accepts; anything else falls through to an ordinary roll, untouched. Your stats are shown alongside the result either way.

A Second Chance

After a roll the tale **waits** rather than moving straight on. If you have a reroll to spend you are offered one, alongside a **Carry on** button:

`Reroll (2 left)`	*(spend one and roll again)*
`▶ Carry on`	*(take the result and continue)*

Only one reroll is offered per roll — the second result stands, and the tale moves on. If you have none left, or the scene was an ending, it continues as before without stopping to ask. Your progress is saved while it waits, so nothing is lost if you take a moment to decide.

*In **/activity demo** the reroll is **free** and always offered, even with none left — a dry run should not quietly cost a player their real rerolls just to see what the button does.*

Each scene posts with a button per stat it accepts. A press rolls with the presser's own sheet, honours a Hero's signature stat, and lands in the roll audit like any other roll. Writing and deleting always need a Game Master; starting one is GM-only until **/config mechanics activities players:true**.

Quizzes

A quiz is an activity a Game Master sets from the question bank: a question, then a button per answer — or a box to type into — marked right or wrong on the spot, with the explanation after, and a score at the end. The bank prefers questions you have never been asked, so a second sitting is a different quiz.

Writing One

/activity create opens a paste window for your script; long scripts (over 4000 characters) travel as an attached **.txt** on its **file:** option instead. Pasting the script straight into any channel the bot can read works too. Every route is the same: the script starts with **[ACTIVITY] Name**, re-using a name replaces that activity, and the whole thing is checked before anything is saved.

```
[ACTIVITY] Fishing
TALLY renown                                  (a running total, paid out at the end)

SCENE find
SAY Find a spot to set up.
ROLL wis DC15                                 (a difficulty to beat)
  PASS -> cast
  FAIL ONE OF                                 (picks a different line each loop)
    This spot does not look all too lucky...
    They are not biting here today...
  FAIL -> find                                (loops back)

SCENE cast
ROLL str|dex|wis                              (the roller picks the stat)
  1-5   Small fry.  -> fight_small            (ranges on the total)
  16+   A monster!  -> fight_extra

SCENE fight_big
GAUNTLET str|con 14 12 10                     (three rolls, each harder to fail)
  NAT20 It leaps aboard. -> caught
  NAT1  Your line snaps. -> restring
  PASS -> caught
  FAIL -> find

SCENE caught
```

```
GAIN renown 3                          (banked on arrival)
CHOICE                                 (buttons, no dice)
  Carry on      -> cast
  Call it quits -> depot

SCENE depot
END TALLY                              (pays the tally out as renown)
```

What Each Line Does

Line	Meaning
SCENE name	A step. The first one is where a run begins.
SAY ...	Narration. Runs over as many lines as you like.
AS Cave Orc	Speak this scene in an NPC's voice, through their webhook.
ROLL str	Ask for a roll. str|dex|wis lets the roller choose.
ROLL wis DC15	Beat 15 for PASS, else FAIL.
GAUNTLET str|con 14 12 10	A run of rolls, each with its own DC. All must pass.
GAUNTLET 14:str 12:str|con 10:dex	The same, but a different check at every step.
1-5 text -> scene	Branch on the roll total. 16+ is open-ended.
PASS / FAIL text -> scene	Branch on the outcome band.
BAND ONE OF	Indented lines below become random variants.
NAT20 / NAT1 text -> scene	Overrides everything else.
CHOICE	Buttons instead of dice. Options are label -> scene.
TALLY renown	Names a running total, declared once at the top.
GAIN renown 3	Adds to the tally when a player arrives at this scene.
END	Finishes. END TALLY pays out, merits:2 awards merits,
	rewards:a silver key is announced for you to hand out.

Without a **DC** or ranges, outcomes fall back to the same bands a **?** check uses: **CRIT** a natural 20, **PASS** 15+, **PARTIAL** 10–14, **FAIL** under 10, **FUMBLE** a natural 1. You need not define all of them — a crit falls back to PASS, a fumble to FAIL.

*Validation refuses a branch pointing at a scene that does not exist, a duplicate scene name, a scene with no roll, choice or ending, a roll on something that is not a stat, a gauntlet longer than eight, and a **GAIN** with no **TALLY** — each with the reason, so a run can never dead-end.*

The Anatomy of a Script

The table above is the quick reference; this is the long walk. Each part of a script, what it does at the table, and the shapes it can take — every example below is a working fragment you can lift.

The Header

```
[ACTIVITY] Fishing                    (the first line — everything above it is ignored)
```

A script begins at **[ACTIVITY] Name** and the name is how everything else refers to it — **/activity run name:Fishing**, **/activity show**, **/activity set**. Writing a script with a name already in use **replaces** that activity in one motion; there is no separate edit-and-save. (**[STORY]** is accepted as an older spelling of the same header.)

SCENE — the Steps of the Tale

```
SCENE find                            (a step, named)
  ...
SCENE cast                            (the next — order on the page does not matter)

  PASS -> cast                        (branches name their destination)
```

A scene is one step: it narrates, asks for something — a roll, a choice — and branches. **The first scene in the script is where every run begins**; after that, order on the page means nothing, because movement is only ever by name: **-> cast** goes to **SCENE cast** wherever it was written. Names are single words, and every branch must point at a scene that exists — validation refuses the script otherwise, so a run can never walk off the map.

SAY — Narration

```
SAY You survey the area for a likely spot,
reading the water for the promise of fish...   (bare lines continue the narration)
```

Whatever follows **SAY** is spoken by the bot when a player arrives at the scene, and it runs over as many lines as you like — a bare line under a SAY simply continues it. Emoji, **bold** and *italics* come through as typed.

AS — an NPC's Voice

```
SCENE gatekeeper
AS Cave Orc                           (this scene speaks through the Cave Orc)

SAY Halt! Who goes there?
```

Give a scene an **AS** line naming a registered NPC and its narration is delivered through that NPC's webhook — their name, their avatar — exactly as **/npc say** would. The rest of

the scene (the roll, the buttons) behaves as normal; only the voice changes.

ROLL — Asking for Dice

```
ROLL wis                          (one stat — outcome read off the bands)

ROLL str|dex|wis                  (the roller chooses which to use)

ROLL wis DC15                     (a difficulty: 15+ is PASS, under is FAIL)
```

A scene with a **ROLL** posts a button per stat it accepts; pressing one rolls **1d20 + that stat from the roller's own sheet**, honouring a Hero's signature advantage and landing in the roll audit like any other roll. Offer several stats with **|** and the choice belongs to the player — a climb might take **str|dex**, brute force or nimbleness.

Typing answers too: **wis** alone, or **wis I read the currents where the reeds thin** to roll and roleplay in one breath — the flavour is printed with the result. A typed stat only answers the scene if it is one that step accepts; anything else falls through to an ordinary roll and the tale keeps waiting.

With a **DC**, the total decides: meet or beat it for **PASS**, fall short for **FAIL**, and only those two bands apply. Without one, the outcome falls onto the full five-band ladder described under Outcome Bands below.

Ranges — Branching on the Number Itself

```
ROLL str|dex|wis

  1-5   Something small brushes the line. -> fight_small

  6-10  A decent weight takes the bait.   -> fight_medium

  11-15 The rod bends hard.               -> fight_big

  16+   The reel screams. Extraordinary!  -> fight_extra   (open-ended top)
```

Instead of bands, a roll can branch on the **total itself** — each line gives a span, the text to speak, and where to go. **16+** is open at the top. Ranges make graded results natural: the same cast, four sizes of fish. A natural 20 or 1 still overrides a range if a **NAT20** or **NAT1** line is present.

GAUNTLET — a Run of Rolls

```
GAUNTLET str|con 14 12 10         (three checks: DC 14, then 12, then 10)

GAUNTLET 14:str 12:str|con 10:dex (or a different test at every step)

  PASS -> caught                  (all steps passed)

  FAIL -> find                    (any step failed)
```

A **GAUNTLET** is several rolls in a row and **all of them must pass**. The first shape names the stats once and lists the DCs; the second gives each step its own DC and its own stats,

so a chase can open on raw speed and end on wits. Eight steps is the ceiling. The scene's **PASS** branch fires only when the whole run is survived; **FAIL** fires at the first stumble — and a **NAT20** or **NAT1** on any step can cut straight out of the sequence.

Outcome Bands — CRIT, PASS, PARTIAL, FAIL, FUMBLE

```
ROLL wis
  CRIT    A revelation! -> shortcut          (natural 20)
  PASS -> onward                             (total 15+)
  PARTIAL You manage, at a cost. -> onward   (total 10–14)
  FAIL -> lost                               (total under 10)
  FUMBLE  Utter disaster. -> ditch           (natural 1)
```

Without a DC or ranges, a roll's outcome lands on the same ladder a **?** success check uses: **CRIT** on a natural 20, **PASS** at 15 or more, **PARTIAL** from 10 to 14, **FAIL** below 10, **FUMBLE** on a natural 1. You need not write all five — **a missing CRIT falls back to PASS, a missing FUMBLE to FAIL**, and a missing PARTIAL rides with FAIL — so the common pair **PASS / FAIL** is a complete scene on its own. Each band line may carry text to speak, a destination, or both.

ONE OF — Variety on a Loop

```
  FAIL ONE OF
    This spot doesn't look all too lucky...   (indented bare lines are the
                                               variants)
    Not a bite. Time to move on.
    Try, try, try again!
  FAIL -> find                                (the same band still branches)
```

A band that ends in **ONE OF** collects the indented lines below it and speaks a **different one each time** — the cure for a looping scene repeating itself word for word. The block ends at the next directive or band name, so **FAIL ONE OF** followed later by **FAIL -> find** reads naturally: varied words, one destination.

NAT20 and NAT1 — the Die Overrides Everything

```
  NAT20 It practically leaps into your hands. ->
caught
  NAT1  The line snaps. -> restring
```

A **NAT20** or **NAT1** line answers the **natural die**, before modifiers — and it beats a DC, a range, and every band. Inside a gauntlet it cuts out of the sequence on the spot. Use them for the moments that should feel like fate regardless of the arithmetic.

CHOICE — Buttons Instead of Dice

```
CHOICE
  Keep fishing   -> cast          (label -> destination)
  Call it a day  -> depot
```

A **CHOICE** scene rolls nothing: it posts one button per line, the label as written, and pressing one moves the run to its destination. Buttons belong to the run's owner — someone else pressing yours is refused — so several players can sit in the same channel, each at their own crossroads. A scene may narrate with SAY first and then offer the choice.

TALLY and GAIN — Keeping Count

```
TALLY renown                    (declared once, near the top)

SCENE caught
GAIN renown 3                   (banked each time a player arrives
                                 here)
```

Declare a **TALLY** once and the run carries a counter. Every scene with a **GAIN** adds its amount **when a player arrives there** — loop through the catch five times and it banks five times. The count is per-run and per-player, shown as it grows, and it pays out only if an ending says so; abandoning a run with **/activity stop** forfeits it. A **GAIN** without a declared **TALLY** is refused at validation.

END — Finishing, and What It Pays

```
END                             (the tale simply ends)
END TALLY                       (pays the counter out as renown)
END merits:2                    (awards 2 merits to the player)
END rewards:a silver key        (announced for you to hand out)
END TALLY merits:2 rewards:a fine rod   (all three at once)
```

A scene with **END** stops the run — after its SAY, so an ending can still speak. What follows the word is the payout: **TALLY** converts the banked counter into renown, **merits:** awards merits **automatically**, with the activity's name on the record, and **rewards:** is free text — an item, a favour, a rumour — **announced** for the Game Master to hand over by hand, exactly as quest rewards work. All three combine on one line, in any order after END.

*The parts compose. A scene may SAY in an NPC's voice, ROLL with a choice of stats, override on the naturals, vary its failures with ONE OF, and GAIN on arrival — the fishing demo uses nearly every part in forty lines, and **/activity show name:Fishing (demo)** after running **/activity demo** reads it back scene by scene as a worked answer key.*

Quizzes & the Question Bank

A quiz is an activity underneath, so everything here rides the machinery in **Activities** — buttons, scoring, handing a run to a named player. What is different is the **bank**: questions written once, kept, and drawn on for ever after. Read the walkthrough first; the rest is reference.

Running a Quiz — Step by Step

1. Give the bank a home. Make a **Forum** channel — call it anything, *quiz-bank* does nicely — then run **/config channels quizforum channel:#quiz-bank**. You only ever do this once. Questions written before you do it are still saved; they just have no posted copy yet.

2. Write your first question. Run **/quiz add category:Orders**. The category is whatever you want to group by — Orders, History, Law. Type a new one and it appears in the dropdown from then on. A window opens with five boxes:

The question	*(How many knight orders are there?)*
Answer 1	*(5)*
Answer 2	*(* 8)*
Answer 3	*(12)*
Answer 4	*(15+)*

The * marks the right answer — one star, and only one. Leave answers 2 to 4 blank and it becomes a question they *type* the answer to, with answer 1 as what you will accept. Marking is forgiving: capitals, accents, punctuation and a leading “the” are all ignored.

3. Add the trimmings, if you want them. On the command, before the window opens: **tags:** for finer sorting (heraldry, ranks — comma-separated), **difficulty:** easy, normal or hard, and **explain:** a line shown after the answer, which turns a test into teaching.

4. Check what you have. /quiz list shows every category, how many questions are in each, and a link to its thread. **/quiz show id:7** reads one back; **/quiz remove id:7** deletes it and its posted copy.

5. Set the quiz. /quiz start on its own draws five from the whole bank for you to try. In practice you will want a few of these:

category: / tag: / difficulty:	*(draw only from part of the bank)*
count:	*(how many questions — five by default)*
player:	*(hand it to someone to sit; the buttons are theirs)*
mode:	*(tally (carry on) or retry (same question until right))*
pass: / merit:	*(how many right passes, and what passing earns)*
pool:	*(fresh first, only new, any, or revision)*
save: / set:	*(remember this draw under a name, and reuse it)*

6. What the player sees. The question, then a button per answer — or a box to type in. Each answer is marked right or wrong on the spot, your explanation follows it, and the run ends with a score: **Score: 4/5 (80%)**, and whether they passed.

*A draw normally prefers questions that player has never been asked, so a small bank still feels fresh. **pool:Revision** flips that on its head and asks only the ones they got wrong before — which*

is how you send someone away to learn and then test the same ground.

7. Do it again next week. Add **save:Induction** the first time, and after that **/quiz start set:Induction player:@newcomer** repeats the whole thing — same filters, same count, same pass mark — with fresh questions drawn each time.

The Question Bank

Questions can be banked once and drawn on for ever after. **/quiz add category:Orders** opens a window with five boxes — the question, then up to four answers — and a * in front of an answer marks it correct. Fill only the first and it becomes a typed question with that as the accepted reply. Add **tags:** for finer sorting, **difficulty:**, and **explain:** for a line shown after the answer.

Set **/config channels quizforum** and the bank gets a readable home: a thread per category, opened as questions are written. The forum is a copy for reading — the bot keeps its own, which is what makes a draw instant and the dropdowns possible.

`/quiz add category:Orders tags:heraldry`	*(write one — a window opens)*
`/quiz list [category:] [tag:]`	*(what is in the bank, and where)*
`/quiz show id:7`	*(read one in full)*
`/quiz remove id:7`	*(delete it, and its posted copy)*
`/quiz start count:10 category:Orders pass:8 merit:1`	*(draw ten from Orders; eight right passes and earns a merit)*
`/quiz start set:Induction player:@new`	*(set a saved draw for a named player)*
`/config channels quizforum channel:#forum`	*(where the bank is posted to read)*

A draw prefers questions that player has never been asked, and only repeats when the bank runs dry. Both the order of the questions and the order of the answers within each are shuffled, so nobody learns that the answer is always B. **mode:** chooses tally or retry, **pass:** sets how many right counts as a pass, **merit:** is what passing earns, **player:** hands it to someone to sit, and **save:** remembers the whole draw under a name to set again later. **pool:** decides which questions they may be asked at all — fresh first, only ones new to them, any at all, or revision: only the ones they got wrong before.

Quizzes

An activity can ask questions and mark the answers. **ASK** makes a scene a question, **ANSWER** lists what counts as right (separate several with **|**), and **RIGHT ->** and **WRONG ->** say where each outcome leads. The player types their answer into a box, so a quiz can want words rather than a pick from four. Marking is forgiving: capitals, accents, punctuation, extra spaces and a leading “the” or “a” are all ignored. The score is kept and read out at the end.

`[ACTIVITY] Kalidale History`	
`SCENE q1`	
`SAY Who founded the White Order?`	
`ASK`	*(the player types their answer)*
`ANSWER Artorius|Artorius of Lyssa`	*(any of these count)*

```
HINT He is still with us.          (shown under the question)
RIGHT -> q2
WRONG -> q1                        (send them round again, or onward — your call)
```

Multiple choice works too: put a * in front of the right option in a CHOICE block and the engine marks it. **QUIZ tally** lets every answer carry on and reads the score out at the end; **QUIZ retry** sends a wrong answer back to the same question until they get it; **QUIZ silent** tells them nothing as they go — each answer is simply recorded — and marks the whole paper at the end, question by question, with your explanations under the ones they missed. Set it on the script, or pick it with **mode:** on /quiz start. Try it with **/activity demo which:Kalidale Lore** — five questions, built in.

```
QUIZ tally                         (or QUIZ retry, at the top of the script)

SCENE q1
SAY How many knight orders are there?
CHOICE
  A — 5 -> q2
  * B — 8 -> q2                    (the star marks the right answer)
  C — 12 -> q2
```

◈

Scrolls, the Bot’s Voice, and Fixing What Was Said

Scrolls

/gm scroll
a modal for title and body; posts the parchment as an image everyone can see plus a PDF with the writing woven invisibly inside — the PDF survives Discord, so scrolls travel between servers on their own. file: hands any scroll PDF back: plain text out, plus a readable edition in standard type as image and woven PDF. A file-name field on the modal names both files for archiving; the readable pair inherits it

/config mechanics scrollfont font:<file>
store an .otf/.ttf — the face /gm scroll props are written in; the reply renders a sample line to prove it

/config channels scrollarchive
a library for every scroll’s named PDF — point it at a FORUM and each GM’s scrolls file in their own thread, opened automatically on first use; a plain channel keeps the flat archive

/gm scroll file:<pdf> — *(read a scroll PDF back as text, and unfurl a fresh copy in this server’s font)*

Speaking as the Bot

`/dd message:… header:… user:@a channels:#one #two`
speak AS THE BOT — so a GM who also plays a character never blurs the two. Plain words by default; a bold header only if you write one; posted in every channel you name, or here. Who said it goes to the roll-audit, not the room (GM)

Fixing What Was Said

`Edit (DDice)  — long-press a message → Apps`
edit anything the bot posted — its own messages and NPC speech from /npc say alike. The text opens prefilled, save replaces it, and the roll-audit records what changed (GM)

Only messages the bot itself posted — its own, and NPC speech sent through **/npc say**. The roll audit records what changed.

Keeping the Server

Backups

With a channel set, the database is posted there **every 24 hours** — and each backup **deletes and replaces the last**, so the channel holds exactly one file: the newest. **/gm backup now** takes one on demand and replaces the standing post the same way; with no channel set it comes back to you privately as a one-off copy instead.

`/gm backup now`	*(take one immediately — Game Master)*
`/gm backup auto channel:#backups`	*(automatic backups — Game Master)*
`/gm backup auto channel:#backups hours:12`	*(or a different interval — Game Master)*
`/gm backup auto channel:off`	*(stop them — Game Master)*

*The cycle is measured from the **last backup taken**, not from when the bot last started. A redeploy mid-cycle resumes where it left off, and one that happens while a backup is overdue takes it shortly after boot — so a server that redeploys several times a day still gets its daily file. Switching backups on takes one immediately rather than waiting out the first cycle.*

*The file is a **settled snapshot**, not the live database — taken with SQLite's own **VACUUM INTO**, so a write landing mid-upload cannot produce a torn file, and unused pages are dropped so the attachment is as small as it can be. The new backup is posted **before** the old one is removed, so a failure part-way leaves the previous file in place rather than none at all.*

Destructive actions (/npc delete, /fight end, /quest delete, /char weapon remove) ask for Confirm / Cancel before running.

Publishing These Books

The three books are built outside the bot and committed to a repository. Point the bot at that repository and it will keep **one current post** of all three in a channel of your choosing — when a new build lands, the old post is deleted and replaced rather than the channel filling up with stale copies.

`/config channels docs channel:#gm-books repo:owner/name`	*(set it up)*
`/config channels docs player_channel:#rules`	*(player book alone, posted silently)*
`/config channels docs branch:live path:docs`	*(if not on main, or not at the root)*
`/config channels docs push:true`	*(fetch and repost right now)*
`/config channels docs`	*(what is being watched, and the current post)*
`/config channels docs disable:true`	*(stop watching)*

It looks every **15 minutes**, and **pings the Game Master roles** whenever it publishes. Checking is cheap — it asks GitHub for the file versions rather than downloading, so a check that finds nothing new costs three small requests.

Give it a **player_channel** as well and the **player book alone** is kept current there — posted **silently**, with no role pinged and no notification raised, so a reference channel stays up to date without nagging anyone. It replaces its own previous post the same way.

*The new post goes up **before** the old one comes down, so a failure part-way leaves the channel with a copy rather than none at all. The repository must be public and hold the eight books under the names this script gives them: **DDice-Commands-Player.pdf**, **DDice-Commands-GameMaster.pdf** and **DDice-Commands-Parchment.pdf** for Knightfall, the same three prefixed **DnD5e-** for 5e, and the two **DDice-Examples** books. A table is handed the set for the rules it plays by, and the Examples either way.*

Updates and the Deploy Mend

Every deploy writes two dated threads into the **gm-updates** forum: **What's new** — what changed, in the GM's terms — and **Deploy mend**, what the boot built or repaired. The player half of the same news goes to the **updates** forum. Each thread is titled by its date, *[11/10/2026] [WEEK 41] Sunday 11th of October 2026 · What's new*, so the newest sits at the top and the week is readable at a glance. The first boot with the forums in place posts the whole back-catalogue, oldest first, and clears the old update posts out of gm-quest-log.

Checks and Reports

`/gm check roster busy:true`	*every player at a glance — HP, and whether a quest or a fight is holding them (red means the next rest will pass them over); busy:true shows only those tied up (GM)*
`/gm check usage`	*(sixty days of command use, quietest first — a report only; nothing is ever removed by it (GM))*
`/gm check migrations`	*(the ledger of one-time migrations — which have run, which are pending, and what each did (GM))*
`/gm check pages user:@a`	*rebuild character pages from scratch — every bot block cleared and re-posted in order; reports what would not delete. Blank does everyone (GM)*
`/gm check portraits`	*move every stored NPC face into its category thread — expired links recovered from history, lost ones named (GM)*

`/gm dicereport`	*(the table's dice health — top rollers, hot and cold d20 hands, nat leaders, the full d20 spread)*

`/gm questwipe runs:true`
delete EVERY quest on the server — confirm-gated; rosters, timelines and book entries go, threads stay. The run ledger and the DM cards' guided counters survive unless runs:true

Test Tools

Trying a feature out usually means inventing a quest or an NPC you then have to tidy out of the world. **/gm test** makes throwaway ones instead. Everything it creates is named **[test]** and can be swept away in one command. It is hidden from players entirely.

`/gm test quest`	*(a quest with you on the party, in this channel)*
`/gm test npc`	*(an NPC with items, standing, rolls and lore already on it)*
`/gm test list`	*(what it has made)*
`/gm test forum`	*(exercise the NPC forums live, tidying up after itself)*
`/gm test clean`	*(delete all of it — asks first)*

The test quest arrives ready to start, so the clock, the reminders, the timeline and the summary can all be exercised in a few minutes. The test NPC arrives with a record already on it, so **/npc sheet** has something to show.

*Cleaning only ever touches rows named **[test]**, and clears their events, summaries, inventory, roll tallies and standing log along with them.*

◈

Reference

What the Stats Do

`/char stat`	*(what each stat is for, in plain words)*

How Rosters Are Ordered

Every list of characters reads the same way: **Heroes first** as their own group, then by **knight order**, then by **class**, then alphabetically. Characters with no order or no class sort after those that have one, rather than jumping the queue.

***NPCs sit below the players**, however senior their order — the two are separate halves of a list rather than interleaved. Within the NPC half the same ordering applies. Mixed lists like **/gm heal global:Everyone** print each group under its own heading.*

Telling the GMs Something

`/feedback send`
tell the GMs what you think — pick a room, score it 1–10, say your piece; they see who wrote it, no one else sees it happened

`/feedback send room:General quest:…`
name the room to go straight to the form (blank opens a menu); name a run you were on or ran and the GMs' card says which one

---

Penned by the DDice Scriptorium · May your rolls be ever in your favour